

1. Building Demo App Chat Client

Static serving with Express

1. Download **Postman**.
2. Buat project baru dengan nama **NodeAppChat**.
3. Click kanan **NodeAppChat** lalu buat directory baru dengan nama **DemoApp**.
4. Click kanan folder **DemoApp** dan buat file baru dengan nama **server.js**.
5. Perlu diingat bahwa **Express** tidak ada pada Node, maka kita perlu install menggunakan npm sebagai third party packages.
6. Buka **terminal** di IntelliJ di sudut kiri bawah atau click **View > Tool Windows > Terminal**.
7. Ketik di terminal: **cd DemoApp**.
8. Ketik di terminal: **npm init --yes**.
9. Selanjutnya install Express dengan ketik: **npm install -s express**. Command **-s** untuk simpan di **package.json**.
10. Cek **package.json**, sudah ada dependencies express.
11. Lalu cek folder **node_modules**, sudah ada folder **express**.
12. Ketik code berikut di **server.js**:

```
var express = require('express');
var app = express(); //set reference variable

app.listen(3000);
```
13. Ketik di terminal: **nodemon .\server.js** atau **nodemon server.js**.
14. Buka browser lalu ketik: **localhost:3000**.
15. Buka developer console dengan mengetik **ctrl+shift+i**.
16. Lalu pilih tab **console**.
17. Dapat respon tapi muncul **error 404** karena tidak ada file yang di host/dipanggil.
18. Buatlah konten static **index.html**.
19. Lalu ketik word: Hallooooo
20. Tambahkan code di file **server.js**:

```
var express = require('express');
var app = express(); //set reference variable

app.use(express.static(__dirname));
app.listen(3000);
```
21. Refresh browser. Sekarang sudah tidak muncul error lagi.
22. Edit **server.js** menjadi:

```
app.use(express.static(__dirname));
var server = app.listen(3000, function () {
  console.log("port server adalah", server.address().port)
});
```
23. Lihat di terminal.
24. Backend sudah jalan, selanjutnya frontend.

Browser app

1. Edit **index.html**:

```
<!doctype html>
<html lang="en">
  <head>
    <title>Belajar NodeJS</title>
  </head>
  <body>

  </body>

</html>
```
2. Agar layout terlihat bagus gunakan library bootstrap: <https://getbootstrap.com/>
3. Lalu click **Get Started**.

4. Copy **CSS** lalu paste ke index.html:

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-beta2/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
BmbxuPwQa2lc/FVzBcNJ7UAyJxM6wuqIj61tLrc4wSX0szH/Ev+nYRRuWlolflfl" crossorigin="anonymous">
```
5. Test di browser, lihat di tab **Network** apakah **css** sudah di load.
6. Copy lagi **JS Bundle** dan paste ke **index.html**:

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-
beta2/dist/js/bootstrap.bundle.min.js" integrity="sha384-
b5kHyXgcpbZJO/tY9U17kGkf1S0CWuKcCD3818YkeH8z8QjE0GmW1gYU5S9FOnJ0"
crossorigin="anonymous"></script>
```
7. Copy jQuery dari <https://jquery.com/download/> lalu paste ke <head>:

```
<script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>
```
8. Setelah script tambahkan code berikut kedalam **index.html**:

```
<div class="container">
  <br>
  <div class="jumbotron">
    <h1 class="display-4">Kirim Pesan</h1>
    <br>
    <input class="form-control" placeholder="Nama">
    <br>
    <button class="btn btn-success">Kirim</button>
  </div>
  <div id="pesan">

  </div>
</div>
```
9. Lihat di browser coba perkecil browser untuk menunjukkan tampilan sudah **responsive**.
10. Lalu dibawah div body buatlah tag script:

```
<script>
  $(function () {
    console.log('loaded');
  });
</script>
```
11. Lihat di browser dan pilih tab **Console**.
12. Lalu modifikasi tag script menjadi:

```
<script>
  $(function () {
    tambahPesan({nama: 'Jurusan', pesan: 'Informatika'})
  });

  function tambahPesan(pesan) {
    $("#pesan").append(`<h4> ${pesan.nama} </h4> <p> ${pesan.pesan} </p>`)
  }
</script>
```
13. Lihat di browser.
14. Selanjutnya buat click event callback untuk button/tombol.
15. Tambahkan id pada tag button:

```
<button id="kirim" class="btn btn-success">Kirim</button>
```
16. Lalu modifikasi tag script menjadi:

```
<script>
  $(function () {
    $("#kirim").click(function () {
      tambahPesan({nama: 'Jurusan', pesan: 'Informatika'})
    });
  });
</script>
```
17. Test di browser.

Get Messages Service

1. Modifikasi **server.js** dengan menambahkan code:

```
app.get('/pesan', function (req, res) {  
  res.send('Hallooo');  
})
```

2. Lalu buka browser dan ketik: **http://localhost:3000/pesan**

3. Modifikasi **server.js**:

```
var pesan = [  
  {nama: 'John', pesan: 'halo john'},  
  {nama: 'Doe', pesan: 'halo doe'}  
]
```

```
app.get('/pesan', function (req, res) {  
  res.send(pesan);  
})
```

4. Test **http://localhost:3000/pesan** di browser, sudah ada pesan array.

5. Tambahkan function berikut di dalam tag script pada file **index.html**:

```
function getPesan() {  
  $.get('http://localhost:3000/pesan', function (data) {  
    console.log(data);  
  })  
}
```

6. Lalu pada button click handler tambahkan code:

```
$(function () {  
  $("# kirim").click(function () {  
    tambahPesan({nama: 'Jurusan', pesan: 'Informatika'})  
  });  
  getPesan();  
});
```

7. Buka: <http://localhost:3000/> dan lihat di dev tool, pastikan tidak ada error.

8. Lihat tab **Console**, sudah ada pesan yang masuk.

9. Selanjutnya akan dimunculkan di tag div.

10. Edit function **getPesan()**:

```
function getPesan() {  
  $.get('http://localhost:3000/pesan', function (data) {  
    data.forEach(tambahPesan);  
  })  
}
```

11. Selanjutnya akan dicoba untuk post.

12. Install **Postman**.

Post Messages Service

1. Buka file **server.js**.

2. Copy **app.get** dan paste ke bawahnya lalu ubah menjadi:

```
app.post('/pesan', function (req, res) {  
  console.log(req.body);  
  res.sendStatus(200);  
})
```

3. Buka **Postman**, masukkan **localhost:3000/pesan**

4. Ganti **GET** menjadi **POST**.

5. Click tab **Body** dan click radio button **raw**

6. Dropdown ganti ke **JSON (application/json)**

7. Ketik message object berikut:

```
{"nama": "John", "pesan": "Hellooooo dari Postman"}
```

8. Lalu click **Send** dan kita dapat respon **200 OK**.

9. Buka terminal dan muncul **undefined**.

10. Penyebabnya karena express tidak memiliki dukungan built-in untuk parse body.

11. Buka terminal baru dengan click tanda plus.

12. Pastikan install di folder dengan ketik **cd DemoApp**.

13. Ketik di terminal: **npm install -s body-parser**

14. Lalu tambahkan di **server.js**:

```
var express = require('express');
var bodyParser = require('body-parser');
var app = express(); //set reference variable

app.use(express.static(__dirname));
app.use(bodyParser.json());
```

15. Lalu tutup terminal **Local(2)** dan buka terminal yang nodemon sedang berjalan.

16. Buka **Postman**, click lagi Send dan pastikan status **200 OK**.

17. Lihat di terminal, sudah ada pesan dari Postman.

18. Ubah **app.post** menjadi:

```
app.post('/pesan', function (req, res) {
    pesan.push(req.body);
    res.sendStatus(200);
})
```

19. Click **Send** di Postman dan pastikan 200 OK.

20. Refresh **http://localhost:3000/** dan sudah muncul pesan dari Postman.

21. Selanjutnya kita akan post tidak dari Postman tapi dari browser.

22. Edit **index.html** dengan copy function **getPesan()** dan paste di bawahnya menjadi:

```
function postPesan(pesan) {
    $.post('http://localhost:3000/pesan', pesan, function (data) {
        data.forEach(tambahPesan);
    })
}
```

23. Lalu modifikasi click event:

```
$(function () {
    $("# kirim").click(function () {
        var pesan = {}
        postPesan(pesan);
    });
    getPesan();
});
```

24. Tambahkan id name pada input tag Nama dan tambahkan juga textarea:

```
<br>
<input id="nama" class="form-control" placeholder="Nama">
<br>
<textarea id="pesanKirim" class="form-control" placeholder="Pesan"></textarea>
<br>
```

25. Modifikasi click event:

```
$(function () {
    $("# kirim").click(function () {
        var pesan = {nama: $("#nama").val(), pesan: $("#pesanKirim").val()}
        postPesan(pesan);
    });
    getPesan();
});
```

26. Modifikasi function **postPesan()** menjadi:

```
function postPesan(pesan) {
    $.post('http://localhost:3000/pesan', pesan);
}
```

27. Coba jalankan di browser dengan mengisi nama, pesan, click Kirim.

28. **Refresh** halaman **http://localhost:3000/pesan**. Muncul undefined.

29. Kita debug dengan modifikasi **app.post** di **server.js**:

```
app.post('/pesan', function (req, res) {
    console.log(req.body);
    pesan.push(req.body);
    res.sendStatus(200);
})
```

30. Coba posting lagi dari browser.
31. Lihat terminal. Ternyata kosong. {}.
32. Tambahkan **app.use** berikut:


```
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({extended: false}));
```
33. Dan hilangkan **console.log** dari **app.post**:


```
app.post('/pesan', function (req, res) {
  pesan.push(req.body);
  res.sendStatus(200);
})
```
34. Hilangkan data json pada **var pesan** di **server.js**.


```
var pesan = []
```
35. Coba posting lagi dari browser. Lalu **Refresh http://localhost:3000/**.
36. Coba buka 2 tab untuk simulasi chat.
37. **Problem**: harus tetap click refresh.
38. Solusi: gunakan **socket.io**.

Connect to socket.io

1. Buka terminal baru dan ketik **cd DemoApp**.
2. Install **socket.io** dengan mengetik: **npm install -s socket.io**.
3. Buka **server.js** dan tambahkan code berikut:


```
var app = express(); //set reference variable
var http = require('http').Server(app);
var io = require('socket.io')(http);
```
4. Buka **index.html** lalu pada bagian atas tambah script berikut:


```
<script src="/socket.io/socket.io.js"></script>
```
5. Inspect console browser, muncul **error 404**.
6. **Penyebab**: tidak bisa lagi secara langsung akses backend dengan express.
7. Modifikasi var server di file **server.js**:


```
var server = http.listen(3000, function () {
  console.log("port server adalah", server.address().port)
});
```
8. Cek lagi di server, error sudah hilang.
9. Lihat di tab **Network**, sudah ada **socket.io.js**.
10. Modifikasi code berikut di **index.html**:


```
var socket = io();
$(function () {
  $("# kirim").click(function () {
    var pesan = {nama: $("#nama").val(), pesan: $("#pesanKirim").val()};
    postPesan(pesan);
  });
  getPesan();
});
```
11. Lalu di **server.js** dibawah **app.post** tambahkan code:


```
io.on('connection', function (socket) {
  console.log('a user connected');
})
```
12. Refresh browser dan lihat di terminal IntelliJ, muncul: **a user connected**.
13. Modifikasi **app.post** di file **server.js**:


```
app.post('/pesan', function (req, res) {
  pesan.push(req.body);
  io.emit('pesan', req.body);
  res.sendStatus(200);
})
```
14. Tambahkan code berikut di file **index.html** sebelum function **tambahPesan()**:


```
socket.on('pesan', tambahPesan);
```
15. Coba tes di browser, refresh lalu coba post pesan.
16. Buka 2 tab, cek apakah pesan terkirim.

17. Sekarang **socket.io** sudah berjalan.
18. Selanjutnya kita akan simpan pesan ke **MongoDB**.
19. **Tugas:** download dan install **MongoDB Community Edition versi 8**.
20. Instal juga tools **MongoDB Shell**.
21. Demo sebentar MongoDB yang sudah diinstall.
22. Buka **Command Prompt** lalu jalankan MongoDB dengan mengetik: **mongosh**.
23. Ketik: **show dbs**. Akan terlihat list db.
24. Buatlah data base baru dengan mengetik: **use belajar_nodejs**.
25. Masukkan data ke db belajar_nodejs dengan mengetik: **db.coba.insert({"jurusan" : "informatika"})**
26. Sudah berhasil memasukkan satu item ke collection **coba**. Lalu ketik lagi: **show dbs**.
27. Ketik: **show collections**.
28. Sudah ada satu collection yaitu **coba**.
29. Untuk melihat data ketik: **db.coba.find().pretty()**
30. Cara delete database: **use belajar_nodejs**.
31. Lalu ketik: **db.dropDatabase()**.
32. Cek db: **show dbs**.